- PRD `PRD.md` Codex Master Prompt
- Prompt 01–07
- Master Prompt Prompt
-

PRD.md

NodeWitness — Product Requirements Document

1. Document Control

Field	Value
Product	NodeWitness
Tagline	Build AI agents anyone can understand, review, and control.
Short tagline	Every agent step, witnessed.
Product category	Enterprise AI agent governance and no-code infrastructure
Hackathon MVP	RIA Client Mandate Compliance Agent
Primary audience	Non-technical enterprise compliance professionals
Secondary audience	Developers, model-risk teams, auditors, and operations teams
MVP environment	Localhost
Runtime strategy	Local `codex mcp-server` with deterministic Demo Mode fallback
Incremental AI API cost	$0; use the existing authenticated Codex subscription
Product language	English only
Deployment vision	Hybrid SaaS plus private/VPC/on-premises data plane
Status	Hackathon build specification

2. Executive Summary

NodeWitness is an enterprise infrastructure platform for building AI agents that are traceable, reviewable, monitorable, and human-controlled.

A non-technical professional describes a workflow in natural language. NodeWitness turns that description into an interactive agent graph. Every important node can be opened to inspect its inputs, outputs, evidence, structured decision basis, version, status, timestamps, and human modifications.

The hackathon MVP demonstrates one concrete workflow for a U.S. Registered Investment Adviser:

> Monitor a client's Investment Policy Statement, extract private investment restrictions, compare those restrictions against portfolio facts, detect policy changes, re-run only affected compliance checks, and require human approval before any remediation action is executed.

The MVP will demonstrate:

- Natural-language workflow generation.
- Interactive, zoomable agent visualization.
- PDF source versioning and exact-page evidence.
- Continuous monitoring of IPS updates.
- Incremental re-evaluation of affected nodes.
- Compliance conclusions and parallel recommendations.
- Individual Approve, Reject, and Edit controls.
- Human corrections with downstream re-execution.
- Search across the complete workflow and within individual nodes.
- A complete audit trail.
- Safe local execution of approved actions.
- Live Codex execution with a deterministic fallback.

NodeWitness assists compliance professionals; it does not replace their legal judgment or authority.

---

3. Problem Statement

Enterprise professionals want AI productivity gains but cannot safely rely on opaque systems for high-stakes workflows.

Existing agent tools commonly present four problems:

1. Users cannot see how individual steps relate to the final output.
2. Evidence, prompts, versions, and human changes are difficult to inspect.
3. Agents may execute actions before an accountable person understands the consequences.
4. Source changes can silently invalidate earlier conclusions.

Compliance professionals face an additional problem: rules are not limited to general regulations. Each client may have private mandates, restrictions, exceptions, and approval requirements embedded in documents such as an IPS.

Manual review is slow and attention-intensive. Fully autonomous execution is too risky. NodeWitness provides the middle layer: AI-assisted analysis with visible provenance and explicit human control.

---

4. Product Positioning

NodeWitness is not:

- A fully autonomous compliance officer.
- A portfolio management or trading system.
- A generic chatbot.

- A static workflow diagramming application.
- A chain-of-thought viewer.
- A replacement for enterprise systems of record.

NodeWitness is:

- A no-code agent builder for domain professionals.
- A runtime and governance layer for reviewable agents.
- A visual representation of inputs, decisions, evidence, and actions.
- A human-in-the-loop approval and correction interface.
- An event-driven monitoring system that re-evaluates affected work.
- A future integration layer for enterprise systems.

Differentiator

Most agent observability tools are developer-first and inspect an agent after it has run. NodeWitness is operator-first: domain professionals can create, inspect, intervene in, approve, and correct an agent while it is running.

---

5. Target Users

Primary user: Compliance Analyst

Characteristics:

- Understands regulations, policies, and client mandates.
- Has little or no software development experience.
- Needs to understand why the system reached a conclusion.
- Must retain final decision authority.
- Needs defensible records for internal review.

Jobs to be done:

- Turn written client rules into monitorable checks.
- Detect when a policy change affects an existing conclusion.
- Review the evidence behind a finding.
- Decide which remediation steps may proceed.
- Correct an AI interpretation without editing code.
- Export a reviewable record.

Economic buyer: Chief Compliance Officer

Needs:

- Reduced manual review burden.
- Consistent controls.
- Clear accountability.
- Audit-ready records.
- Configurable approval boundaries.

- Private deployment options.

Secondary user: Developer or Model-Risk Specialist

Needs:

- Expandable technical metadata.
- Input and output schemas.
- prompt/model/runtime versions.
- Node-level latency and status.
- Run lineage and replay.
- Integration and deployment controls.

---

6. MVP Scenario

Demo client

`Aurora Family Office`

Source documents

IPS v1

The original IPS allows direct cryptocurrency investments up to 10% of portfolio market value.

IPS v2

The revised IPS reduces the limit to 5% and expands the rule to include direct holdings and look-through exposure through funds, ETFs, and derivatives.

Synthetic portfolio facts

Metric	Value
Direct cryptocurrency exposure	2.0%
Indirect/look-through exposure	6.2%
Total cryptocurrency exposure	8.2%
Largest single-issuer exposure	18.7%
Single-issuer maximum	20.0%
Liquid asset allocation	6.4%
Required liquidity minimum	5.0%

Expected result

Under IPS v1, direct cryptocurrency exposure passes the 10% restriction.

After IPS v2 arrives:

1. NodeWitness detects the new version.
2. The rule-diff node identifies the changed threshold and expanded exposure scope.
3. Only the affected crypto rule branch is re-evaluated.
4. Total look-through crypto exposure of 8.2% is compared with the new 5% maximum.
5. The status changes to `Review Required`.
6. A remediation case is proposed as the primary action.
7. Additional recommendations are displayed in parallel.
8. Every recommendation supports Approve, Reject, and Edit.
9. Nothing executes until the user approves it.

---

7. Product Goals

Hackathon goals

- Demonstrate the complete story in under four minutes.
- Make the agent's behavior understandable without explaining source code.
- Show a visually strong before-and-after policy update.
- Prove that humans retain decision authority.
- Run reliably without additional paid API access.
- Survive connectivity or Codex runtime failure through Demo Mode.

Success metrics

Metric	MVP target
Workflow generation feedback	Under 2 seconds in Demo Mode
Source-update detection	Under 2 seconds
Visible affected-node re-run	Under 5 seconds in Demo Mode
Search result response	Under 200 ms
Approval granularity	Every proposed action
Evidence coverage	Every compliance conclusion
Audit coverage	Every decision and modification
Demo completion rate	100% across three rehearsals
External API cost	$0

---

8. MVP Scope

P0 — Required

- One RIA compliance workspace.
- English-only interface.

- Natural-language workflow description.
- Preset workflow generation in Demo Mode.
- Live graph generation through Codex when available.
- Interactive graph with pan, zoom, fit-to-view, selection, and expansion.
- Bundled synthetic IPS v1 and v2 PDFs.
- PDF upload interface optimized for the bundled documents.
- IPS version history and text diff.
- Evidence links to PDF page and quoted text.
- Automatic re-trigger after a source-version event.
- Re-run only affected nodes.
- Compliance status and structured decision basis.
- Primary remediation case recommendation.
- Parallel attention points and action recommendations.
- Individual Approve, Reject, and Edit controls.
- Node-level correction and downstream re-run.
- Global workflow search.
- Search inside an expanded node.
- Audit timeline.
- Local remediation case creation.
- Local compliance memo generation.
- Local adviser notification draft generation.
- JSON audit export and Markdown memo download.
- Live Codex and Demo Mode status badge.
- Automatic or one-click fallback to Demo Mode.
- Responsive layout suitable for a laptop presentation.

P1 — Implement only after P0 is stable

- Keyboard shortcuts.
- Animated edge traversal.
- Side-by-side PDF viewer.
- Persist session state in `localStorage`.
- Copy-to-clipboard actions.
- Reduced-motion accessibility mode.
- Demo reset button.

Explicitly out of scope for MVP

- Arbitrary IPS extraction accuracy.
- Production authentication.
- Multitenancy.
- Real customer data.
- Real trading integration.
- Email delivery.
- Jira, ServiceNow, or other ticket execution.
- Enterprise RBAC.
- Production databases.

- Cloud deployment.
- Billing.
- Mobile optimization.
- Multiple industry templates.
- Full legal or regulatory coverage.

The upload interface may accept arbitrary PDFs, but only the bundled IPS v1/v2 path is guaranteed for the demo.

---

9. Core User Journey

Journey A — Build the agent

1. The user opens the RIA Compliance workspace.
2. The user enters:

   ```
   Monitor client IPS restrictions, compare them with portfolio exposure,
   explain every compliance conclusion with evidence, and require human
   approval before remediation.
   ```

3. The user selects `Generate Agent`.
4. NodeWitness generates and displays the workflow graph.
5. The interface explains that the workflow is editable and inspectable.

Journey B — Review the original policy

1. The user loads IPS v1.
2. NodeWitness extracts the crypto restriction.
3. The user searches for `crypto`.
4. Matching nodes are highlighted.
5. The user opens the compliance-check node.
6. NodeWitness displays inputs, output, evidence, decision basis, version, and status.
7. The conclusion is `Pass`.

Journey C — Detect a policy update

1. Monitoring is visibly active.
2. The user uploads or triggers IPS v2.
3. NodeWitness registers a `source.version.changed` event.
4. The policy-diff node displays the old and new rule.
5. Affected nodes pulse or change to `Re-running`.
6. Unaffected nodes remain unchanged.
7. The conclusion becomes `Review Required`.

Journey D — Human review and remediation

1. NodeWitness proposes a remediation case.

2. Parallel recommendations appear.
3. The user edits the remediation description.
4. The edited node is marked `Human Modified`.
5. Downstream outputs are re-generated.
6. The user approves the remediation case.
7. The user rejects or edits another recommendation.
8. Only approved actions execute.
9. Created artifacts appear in the Outputs panel.
10. The audit timeline records every event.

---

10. Functional Requirements

FR-1: Natural-Language Agent Builder

- Provide a prominent natural-language input.
- Include a prefilled demo prompt.
- Generate a graph schema consisting of nodes and edges.
- Demo Mode must generate the approved deterministic graph.
- Live Mode may use Codex to generate the same validated JSON schema.
- Invalid model output must never break the UI; use the deterministic graph instead.

FR-2: Interactive Workflow Graph

Every node must contain:

- Node ID.
- Label.
- Business-friendly description.
- Node type.
- Status.
- Input summary.
- Output summary.
- Evidence count.
- Version.
- Last-run timestamp.
- Whether it was modified by a human.

Required interactions:

- Mouse pan.
- Wheel or control zoom.
- Fit-to-view.
- Select node.
- Expand node details.
- Collapse details.
- Center on search result.
- Visually distinguish affected, running, completed, review-required, and human-modified nodes.

FR-3: Expandable Node Inspector

The inspector must show progressively disclosed sections:

1. Summary.
2. Inputs.
3. Outputs.
4. Evidence and source citations.
5. Decision basis.
6. Technical metadata.
7. Run history.
8. Human modification field.

Technical metadata may include:

- Runtime mode.
- Model/runtime label.
- Prompt template version.
- Input schema version.
- Output schema version.
- Thread ID when available.
- Duration.
- Start and completion timestamps.

Do not display private chain-of-thought. Display concise structured decision basis and evidence instead.

FR-4: Global Search

- Search node labels, descriptions, rule text, evidence excerpts, inputs, outputs, and IDs.
- Highlight all matching nodes.
- Display match counts.
- Clicking a result centers and opens the matching node.
- Matching substrings must be highlighted in the inspector.
- Clearing the query restores the normal graph.

FR-5: Node-Level Search

- When a node is open, provide a second search field.
- Search only within the current node’s visible and expandable data.
- Highlight the matching field or evidence excerpt.
- Show a no-results state.

FR-6: IPS Versioning and Diff

- Preserve source name, source version, upload time, effective date, and document hash.
- Show IPS v1 and IPS v2 in version history.
- Display old rule text and new rule text.
- Highlight:
    - Threshold change from 10% to 5%.
    - Scope expansion to look-through exposure.

- Provide page number and evidence excerpt for each version.
- Never overwrite the original version.

FR-7: Monitoring and Automatic Re-evaluation

For the MVP, continuous monitoring means that NodeWitness listens for source-change events inside the localhost application.

- A PDF upload or demo update emits `source.version.changed`.
- The UI must show `Monitoring Active`.
- The event triggers an impact analysis.
- Only affected nodes enter `Queued` or `Running`.
- Unaffected nodes retain their previous run identifier and status.
- The UI displays the most recent monitoring timestamp.
- External source connectors are roadmap items.

FR-8: Compliance Evaluation

The conclusion schema must include:

- Status: `Pass`, `Attention`, `Review Required`, or `Insufficient Evidence`.
- Rule ID.
- Rule version.
- Observed value.
- Threshold.
- Unit.
- Decision basis.
- Evidence citations.
- Confidence label.
- Human-review requirement.

The crypto outcome after IPS v2 must be `Review Required`, not an automatically executed trade instruction.

FR-9: Recommendations

The revised-policy run must create:

1. Primary recommendation: create a remediation case.
2. Draft adviser notification.
3. Generate compliance memo.
4. Validate or refresh look-through exposure data.
5. Optional monitoring recommendation for near-limit concentration.

Every item must independently support:

- Approve.
- Reject.
- Edit.
- View evidence.

- View audit history.

No batch approval is required.

FR-10: Human Correction

- Every key decision node must expose `Suggest a correction`.
- Preserve the original output.
- Store the human instruction as a new immutable audit event.
- Mark the node as `Human Modified`.
- Re-run only its downstream dependent nodes.
- Show the relationship between original output, correction, and revised output.
- Never silently replace historical data.

FR-11: Approval-Gated Execution

Only approved actions may execute.

MVP action handlers:

- Create a local remediation case.
- Generate and save a compliance memo.
- Generate an adviser notification draft.

Rejected actions must remain visible with their rejection status.

Edited actions must retain the original and edited values.

FR-12: Audit Trail

Record:

- Source upload.
- Source-version detection.
- Rule extraction.
- Rule diff.
- Node start and completion.
- Compliance conclusion.
- Evidence citation.
- Recommendation creation.
- Approval.
- Rejection.
- Edit.
- Human correction.
- Re-run.
- Artifact creation.
- Runtime-mode change.

Each audit event requires:

- Event ID.

- Timestamp.
- Actor.
- Event type.
- Target node or action.
- Before value when applicable.
- After value when applicable.
- Run ID.
- Source version.

FR-13: Runtime Modes

Live Mode

- Connect to a locally authenticated `codex mcp-server`.
- Do not require an OpenAI API key.
- Use the user's existing Codex login.
- Run locally through the MCP stdio transport.
- Request structured JSON.
- Validate output before rendering.
- Use read-only sandboxing for document-analysis calls.

Demo Mode

- Use deterministic fixture responses.
- Match the Live Mode response schema.
- Require no network access.
- Clearly display `Demo Mode`.
- Support the complete end-to-end presentation.

Fallback behavior

- Start with Live Mode when the preflight succeeds.
- If the preflight or request times out, explain the fallback in a non-alarming toast.
- Switch to Demo Mode without losing the current workflow.
- Never leave the presentation in a broken loading state.

The MCP design should follow the documented local `codex mcp-server` interface and use its `codex` and, when needed, `codex-reply` tools. See the official Codex MCP guide.

---

11. Information Architecture

Top bar

- NodeWitness logo and name.
- Workspace: `Aurora Family Office — Client Mandate Compliance`.
- Runtime badge: `Live Codex` or `Demo Mode`.
- Monitoring badge.
- Reset Demo.

- Export Audit.

Left panel

- Natural-language agent builder.
- Source documents.
- Version history.
- Monitoring event controls.

Center canvas

- Interactive workflow graph.
- Global search.
- Search result counter.
- Zoom and fit controls.
- Run and re-run animations.

Right inspector

- Node summary.
- Status.
- Inputs and outputs.
- Evidence.
- Node-level search.
- Decision basis.
- Technical details.
- Correction editor.
- Run history.

Bottom or secondary panel

- Findings.
- Proposed actions.
- Approve/Reject/Edit controls.
- Remediation case.
- Generated artifacts.
- Audit timeline.

---

12. Visual Design Direction

Use a polished enterprise operations-console aesthetic.

Recommended palette

- Background: deep navy/charcoal.
- Primary accent: electric cyan.
- Secondary accent: indigo.
- Pass: emerald.

- Attention: amber.
- Review Required: red-orange.
- Human Modified: violet.
- Muted text: cool gray.

Design principles

- Dense enough to feel enterprise-grade.
- Clear enough for a non-technical user.
- Strong visual hierarchy.
- Minimal decorative animation.
- Use motion to communicate execution and impact.
- Keep status labels visible without relying only on color.
- Use subtle grid lines on the graph canvas.
- Avoid generic chatbot styling.

Presentation-critical effects

- Changed nodes pulse briefly.
- Re-running edges animate.
- Search matches glow.
- The old and new rule appear side by side.
- Affected and unaffected branches are visually distinct.
- Approval decisions update immediately.
- The final remediation case appears as a tangible output artifact.

---

13. Recommended Technical Architecture

Stack

- Next.js with App Router.
- React and TypeScript.
- `@xyflow/react` for the interactive graph.
- `@modelcontextprotocol/sdk` for local MCP stdio communication.
- `pdf-lib` only for generating bundled synthetic PDFs.
- Native CSS or CSS modules.
- Browser `localStorage` for demo persistence.
- Static JSON fixtures for deterministic execution.
- Node.js server route for the MCP adapter.

Avoid:

- A separate frontend and backend repository.
- A database.
- Docker for the MVP.
- Authentication.
- Multiple AI-provider abstractions.
- A workflow orchestration framework.

- A vector database.
- External observability services.

Logical architecture

```
Browser UI
  ├── Natural-language builder
  ├── React Flow graph
  ├── Node inspector and search
  ├── Approval controls
  └── Audit and artifacts
          |
          v
Next.js localhost server
  ├── Runtime mode/preflight endpoint
  ├── Codex MCP adapter
  └── Validated response normalization
          |
          +── Live: codex mcp-server over stdio
          |
          └── Fallback: deterministic JSON fixtures
```

Security boundary

- Bind the app to localhost.
- Never read, display, copy, or commit `~/.codex/auth.json`.
- Never commit secrets.
- Never send real client data.
- Treat uploaded PDF content as untrusted.
- Model calls receive only the minimum relevant synthetic content.
- Use read-only sandbox settings for analysis.
- Keep all execution handlers local and reversible.

---

14. Core Data Objects

SourceDocument

```ts
type SourceDocument = {
  id: string;
  name: string;
  type: "IPS" | "PORTFOLIO";
  version: string;
  effectiveDate: string;
  uploadedAt: string;
  hash: string;
  fileUrl?: string;
};
```

EvidenceCitation

```ts
type EvidenceCitation = {
  sourceId: string;
  sourceVersion: string;
  page: number;
  quote: string;
  field?: string;
};
```

WorkflowNode

```ts
type WorkflowNode = {
  id: string;
  type: string;
  label: string;
  description: string;
  status: "idle" | "queued" | "running" | "pass" | "attention" |
          "review-required" | "failed";
  inputs: Record<string, unknown>;
  outputs: Record<string, unknown>;
  evidence: EvidenceCitation[];
  decisionBasis?: string;
  version: string;
  humanModified: boolean;
  technical: {
    runtimeMode: "live" | "demo";
    promptVersion: string;
    runId?: string;
    threadId?: string;
    durationMs?: number;
    startedAt?: string;
    completedAt?: string;
  };
};
```

ProposedAction

```ts
type ProposedAction = {
  id: string;
  type: "REMEDIATION_CASE" | "COMPLIANCE_MEMO" |
        "ADVISER_NOTIFICATION" | "DATA_VALIDATION";
  title: string;
  description: string;
  rationale: string;
  evidence: EvidenceCitation[];
  decision: "pending" | "approved" | "rejected" | "edited";
  originalDescription: string;
  editedDescription?: string;
  executedAt?: string;
};
```

AuditEvent

```ts
type AuditEvent = {
  id: string;
  timestamp: string;
  actor: "system" | "human" | "codex";
  type: string;
  targetId: string;
  runId: string;
  sourceVersion?: string;
  before?: unknown;
  after?: unknown;
  summary: string;
};
```

---

15. AI Output Contract

Live AI output must be strict JSON and conform to the same schema as Demo Mode.

Rules:

- Never request or expose hidden chain-of-thought.
- Ask for concise decision basis.
- Every conclusion must cite evidence.
- Missing evidence must produce `Insufficient Evidence`.
- Model output cannot directly execute an action.
- Recommendations must be separate from conclusions.
- Runtime output must be validated before entering UI state.
- Invalid or incomplete output must fall back to deterministic fixtures.
- Store node-level inputs and outputs, not private internal reasoning.

---

16. Non-Functional Requirements

Reliability

- Demo Mode must run without Codex, network, or external services.
- Every loading state must have a timeout.
- Reset Demo must restore the initial state.
- Three consecutive full rehearsals must succeed.

Performance

- Initial load under three seconds on a modern laptop.
- Search results under 200 ms.
- Graph interactions should remain smooth.
- Demo re-run animations should complete within five seconds.

Accessibility

- Keyboard-focusable buttons.
- Visible focus states.
- Status text in addition to color.
- Sufficient contrast.
- Respect reduced-motion preference.

Privacy

- Synthetic data only.
- No telemetry.
- No external analytics.
- No secrets in client bundles.
- No user data persisted outside the browser for the MVP.

---

17. Acceptance Criteria

The MVP is accepted only if all the following work:

1. The application starts from a documented single command.
2. The user can generate the RIA compliance workflow.
3. The graph supports zoom, pan, fit, selection, and expansion.
4. Searching `crypto` highlights all related nodes.
5. Node-level search highlights evidence within the inspector.
6. IPS v1 produces a passing crypto conclusion.
7. IPS v2 displays the exact rule change.
8. IPS v2 automatically re-triggers the affected branch.
9. Unaffected nodes do not visibly re-run.
10. The revised result is `Review Required`.
11. The primary remediation case appears first.
12. Other recommendations appear simultaneously.

13. Every recommendation supports Approve, Reject, and Edit.
14. No action executes before approval.
15. Editing a key node creates a new audit event.
16. A human correction triggers downstream recomputation.
17. Approved local actions generate visible artifacts.
18. Audit JSON and compliance memo can be downloaded.
19. Live and Demo modes are clearly identified.
20. The complete flow works in Demo Mode without external services.
21. `npm run build` succeeds.
22. The repository contains no credentials or real customer data.

---

18. Four-Minute Demo Script

0:00–0:30 — Problem

“Enterprises want AI agents, but high-stakes users cannot trust a black box. NodeWitness lets non-technical professionals build an agent they can inspect, correct, and control.”

0:30–1:00 — Build

Enter the natural-language workflow description and generate the graph.

Emphasize:

- No code.
- Business-friendly view.
- Expandable technical detail.

1:00–1:35 — Traceability and search

Search for `crypto`.

Open the compliance node and show:

- Inputs.
- Evidence.
- PDF page.
- Decision basis.
- Version and run metadata.

1:35–2:20 — Real-time policy change

Trigger IPS v2.

Show:

- Monitoring event.
- Old-versus-new rule.
- Changed threshold and look-through scope.
- Only affected nodes re-running.

- Result changing from Pass to Review Required.

2:20–3:15 — Human control

Show the remediation case and parallel recommendations.

- Edit one recommendation.
- Approve the remediation case.
- Reject or leave another pending.
- Demonstrate that no batch execution occurs.

3:15–3:45 — Execution and audit

Create the local case and memo.

Open the audit timeline and show:

- Source change.
- AI output.
- Human edit.
- Approval.
- Execution.

3:45–4:00 — Vision

“Today this monitors a client IPS. Tomorrow the same infrastructure governs legal, financial, healthcare, and operational agents across SaaS and private enterprise environments.”

---

19. Development Sequence

Milestone 1 — Demo skeleton

- Scaffold application.
- Add enterprise layout.
- Add deterministic demo state.
- Verify reset and navigation.

Milestone 2 — Graph and evidence

- Create graph.
- Add inspector.
- Add global and node-level search.
- Add PDF evidence navigation.

Milestone 3 — Change monitoring

- Add IPS versions.
- Add diff.
- Add source-change event.
- Add affected-node re-run animation.

Milestone 4 — Human governance

- Add recommendations.
- Add Approve/Reject/Edit.
- Add node correction.
- Add audit trail.
- Add local artifacts.

Milestone 5 — Codex runtime

- Add MCP preflight.
- Add live JSON generation.
- Add timeout and fallback.
- Confirm no API key is required.

Milestone 6 — Demo hardening

- Build verification.
- Empty/loading/error states.
- Demo reset.
- Three rehearsals.
- Final screenshots and README.

P0 must be complete before P1 work begins.

---

20. Post-Hackathon Product Lifecycle

Phase 1 — Design-partner validation

Target:

- Three to five RIA compliance teams.
- Validate rule-authoring, evidence review, and approval workflows.
- Measure time saved and correction frequency.
- Identify required portfolio and document connectors.

Exit criteria:

- Two teams complete a weekly review using NodeWitness.
- More than 80% of findings have usable evidence.
- Users can correct rules without developer assistance.

Phase 2 — Enterprise pilot

Add:

- SSO and RBAC.
- Tenant isolation.
- Encrypted persistent storage.
- Policy and prompt version control.

- Approval-policy configuration.
- Portfolio accounting and document connectors.
- ServiceNow/Jira case creation.
- Email draft and delivery approval.
- Evaluation suite and model-risk reports.

Phase 3 — Hybrid enterprise beta

Architecture:

- SaaS control plane for configuration and administration.
- Customer-controlled data plane for documents, model execution, and traces.
- VPC and on-premises deployment.
- Customer-managed encryption keys.
- Data-residency controls.
- Configurable retention and deletion.
- Full audit export.

Phase 4 — General availability

Add:

- High availability.
- Disaster recovery.
- Enterprise support.
- Usage metering.
- Connector SDK.
- Policy template marketplace.
- Formal security reviews.
- SOC 2 preparation and relevant compliance certifications.
- Operational SLOs and incident management.

Phase 5 — Multi-domain platform

Expand into:

- Broker-dealer supervision.
- Legal contract review.
- Insurance underwriting.
- Healthcare operations.
- Cybersecurity response.
- Procurement and vendor risk.
- Financial controls.

Preserve the same core primitives:

- Source.
- Rule.
- Node.
- Evidence.

- Conclusion.
- Recommendation.
- Human decision.
- Action.
- Audit event.

---

21. Roadmap Integrations

Future approved-action connectors:

- Order-management and trading systems.
- Portfolio accounting systems.
- CRM platforms.
- Email systems.
- Jira.
- ServiceNow.
- Slack and Microsoft Teams.
- Document-management systems.
- Enterprise data warehouses.

All integrations must preserve:

- Least privilege.
- Dry-run mode.
- Explicit approval boundaries.
- Idempotency.
- Complete audit records.
- Reversible actions where possible.

---

22. Key Risks and Mitigations

Risk	Mitigation
Live Codex is slow or unavailable	Deterministic Demo Mode with identical schemas
Arbitrary PDFs parse poorly	Guarantee only bundled v1/v2 documents
UI becomes too complex	Progressive disclosure and one workspace
Users mistake output for legal advice	Persistent human-authority messaging
Judges cannot see incremental re-run	Strong affected-node animation
Agent output lacks evidence	Reject invalid output and use fixture
MCP exposes credentials	Never access or copy the Codex auth file

Risk	Mitigation
Too much scope	No auth, database, external connectors, or production deployment
Demo state becomes inconsistent	Reset Demo and deterministic state machine
Hidden reasoning expectations	Show decision basis and evidence, not chain-of-thought

23. Final Release Checklist

- [ ] All UI text is English.
- [ ] Synthetic data only.
- [ ] IPS v1 and v2 are bundled.
- [ ] Live Mode preflight works.
- [ ] Demo Mode works with no network.
- [ ] Mode badge is always visible.
- [ ] Global and node search work.
- [ ] Evidence page navigation works.
- [ ] Incremental re-run is visible.
- [ ] Approve/Reject/Edit work independently.
- [ ] Human corrections preserve history.
- [ ] Local artifacts are generated.
- [ ] Audit export works.
- [ ] Reset Demo works.
- [ ] README contains exact startup instructions.
- [ ] `npm run build` passes.
- [ ] No API key or authentication file is committed.
- [ ] Three complete demo rehearsals pass.

Codex Prompt Pack

How to use the prompts

Choose one workflow:

- Fastest: run the Master Prompt, then run Prompt 07 for final hardening.
- Safer incremental workflow: run Prompts 01 through 07 in order.
- Do not run the Master Prompt and then blindly repeat Prompts 01–06; inspect the result first.

Every prompt assumes Codex is already opened in the new folder where the repository should be created. No prompt assumes the current conversation's filesystem path.

`PROMPT_00_MASTER.md`

```
You are the lead engineer and product designer for a hackathon project
named NodeWitness.

Work entirely inside the current directory. Do not create or assume any
parent directory. First inspect the current directory. Preserve any
existing user files. If there is no Git repository, initialize one locally,
but do not create a remote repository and do not push anything.

Build a polished localhost MVP that is reliable enough for a live four-
minute demo.

PRODUCT

NodeWitness is an enterprise AI agent governance and no-code infrastructure
product.

Tagline:
“Build AI agents anyone can understand, review, and control.”

The MVP is one English-language demo workspace:

“Aurora Family Office — Client Mandate Compliance”

A non-technical compliance user describes a workflow in natural language.
NodeWitness generates an interactive agent graph. The graph must be
traceable, reviewable, monitorable, searchable, and human-controlled.

TECHNICAL APPROACH

Use the smallest reliable monolithic stack:

- Next.js App Router
- React
- TypeScript
- @xyflow/react
- @modelcontextprotocol/sdk
- pdf-lib only if needed to generate the two synthetic PDF assets
- native CSS or CSS modules
- static JSON/TypeScript fixtures
- localStorage for demo persistence

Do not add:

- a database
- Docker
- authentication
- cloud deployment
- a separate backend project
- Tailwind unless the generated project already uses it
- shadcn
- a workflow orchestration framework
- a vector database
```

```
- analytics or telemetry
- multiple AI provider abstractions

The application must run with:

npm install
npm run dev

It must bind only to localhost by default.

CORE DEMO STORY

Create two synthetic Investment Policy Statement PDFs.

IPS v1:
- Client: Aurora Family Office
- Direct cryptocurrency exposure may not exceed 10% of portfolio market
value.

IPS v2:
- Total cryptocurrency exposure may not exceed 5%.
- Total exposure includes direct holdings and look-through exposure through
funds, ETFs, and derivatives.

Synthetic portfolio:
- Direct crypto exposure: 2.0%
- Indirect/look-through exposure: 6.2%
- Total crypto exposure: 8.2%
- Largest issuer exposure: 18.7%
- Maximum issuer exposure: 20.0%
- Liquid allocation: 6.4%
- Minimum liquidity: 5.0%

IPS v1 must produce Pass for the crypto rule.
IPS v2 must produce Review Required because 8.2% exceeds 5%.

The revised run must create:
1. A primary remediation case proposal.
2. A compliance memo proposal.
3. An adviser notification draft proposal.
4. A look-through-data validation proposal.
5. A near-limit concentration attention item.

Every proposed action must independently support:
- Approve
- Reject
- Edit
- View evidence

Approved actions execute only as safe local operations:
- create a visible local remediation case
- generate a downloadable Markdown compliance memo
- generate an adviser notification draft
```

```
Do not integrate real trading, email, or ticketing systems.

LAYOUT

Create a polished enterprise operations-console interface.

Top bar:
- NodeWitness branding
- workspace name
- Live Codex or Demo Mode badge
- Monitoring Active badge
- Reset Demo
- Export Audit

Left panel:
- natural-language workflow description
- Generate Agent button
- IPS source cards
- version history
- controls to load IPS v1 and trigger/upload IPS v2

Center:
- interactive workflow graph
- global search
- search result count
- pan, zoom, fit-to-view
- clear visual states for idle, running, pass, attention, review required,
and human modified

Right inspector:
- node summary
- node-level keyword search
- inputs
- outputs
- evidence citations
- concise decision basis
- technical metadata
- run history
- “Suggest a correction” editor

Bottom or secondary panel:
- findings
- proposed actions
- Approve/Reject/Edit controls
- generated remediation case
- generated artifacts
- audit timeline

Use a dark enterprise visual style with deep navy, cyan accents, status
colors, clear typography, subtle grid lines, and restrained animations.

Do not make the interface look like a generic chatbot.

WORKFLOW GRAPH
```

```
Include nodes for:

- IPS document source
- portfolio snapshot source
- source versioning
- policy extraction
- version diff
- crypto rule
- exposure calculation
- compliance evaluation
- parallel attention checks
- recommendation generation
- human approval gate
- remediation case execution
- compliance memo output
- adviser notification output

The graph must support:
- mouse pan
- zoom
- fit view
- node selection
- expandable details
- animated affected edges
- visible re-running nodes
- search-result highlighting

GLOBAL AND NODE SEARCH

Global search must index:
- node labels
- descriptions
- IDs
- rule text
- inputs
- outputs
- evidence excerpts

Searching “crypto” must highlight matching nodes. Clicking a result must
center and open the node.

The node inspector must have a separate search field that searches only the
selected node and highlights matching fields and evidence excerpts.

TRACEABILITY

Every important node must expose:

- inputs
- outputs
- evidence citations
- source version
- concise decision basis
```

```
- prompt/schema version
- runtime mode
- timestamps
- duration
- run ID
- thread ID when available
- human-modification status
- previous run history

Do not expose or claim to expose hidden chain-of-thought. Use structured
decision basis and evidence.

SOURCE VERSIONING AND MONITORING

The MVP may guarantee parsing only for the bundled IPS files.

Loading or uploading IPS v2 must emit an internal source.version.changed
event.

Display:
- old rule
- new rule
- 10% to 5% threshold change
- direct-only to look-through scope change
- exact PDF page and evidence excerpt

The event must automatically re-run only the affected crypto branch.
Unaffected nodes must retain their prior run IDs and must not animate as
re-running.

Use a deterministic client-side event/state machine for this if that is the
safest implementation. External connectors are not required.

HUMAN CONTROL

Every proposed action must have its own Approve, Reject, and Edit controls.

Every important decision node must provide “Suggest a correction.”

When the user submits a correction:
- preserve the original value
- create an audit event
- mark the node Human Modified
- re-run only downstream nodes
- display the revised result

No action may execute before explicit approval.

AUDIT

Create an audit timeline containing:
- source upload
- source-version change
- rule extraction
```

```
- rule diff
- node start/completion
- compliance conclusion
- recommendation creation
- edit
- approval
- rejection
- human correction
- re-run
- artifact creation
- runtime-mode change

Support downloading the audit log as JSON.

LIVE CODEX MCP MODE

Implement a thin server-only adapter for the locally authenticated:

codex mcp-server

Use @modelcontextprotocol/sdk with stdio transport. Do not use an OpenAI
API key. Do not read, copy, display, or commit ~/.codex/auth.json.

Use the documented MCP tools:
- codex
- codex-reply only if continuation is necessary

For analysis calls:
- use read-only sandboxing
- use approval policy never
- request strict JSON matching the local fixture schema
- validate returned data before rendering
- set a bounded timeout
- never let invalid output crash the UI

Implement a lightweight preflight/health endpoint.

Default to Live Codex only if preflight succeeds. If preflight or a live
request fails or times out:
- retain all current UI state
- switch to Demo Mode
- display a clear non-alarming notification
- continue with deterministic fixtures

The runtime badge must always show the active mode.

Do not add the OpenAI Agents SDK and do not require an API key. The direct
MCP client is sufficient.

DEMO MODE

Demo Mode must support the entire flow with no network or model dependency.

Live and Demo responses must share one normalized application schema.
```

```
Add Reset Demo so the presenter can return to:
- IPS v1 active
- initial passing result
- no approval decisions
- no generated artifacts
- clean audit timeline

PDF ASSETS

Generate two professional-looking synthetic PDFs and store them in
public/demo.

They must contain page numbers and stable rule text so citations can open
the correct document page. Use an iframe, object element, or simple PDF
viewer link with #page=N. Do not build a custom PDF renderer.

README

Write an English README containing:
- product summary
- architecture
- prerequisites
- installation
- startup
- how to authenticate Codex CLI if necessary
- how Live Mode works
- how Demo Mode works
- exact four-minute demo flow
- troubleshooting
- explicit warning never to commit Codex auth files
- scope and roadmap

QUALITY

Add only the minimum useful automated checks.

At minimum:
- test the IPS v1/v2 policy-diff and compliance decision logic
- test that affected-node selection excludes unrelated nodes
- run lint
- run the production build

Prefer Node’s built-in test capabilities or the smallest test setup that
works with the selected project.

Create .gitignore entries for:
- dependencies
- build output
- local runtime state
- uploads if any
- environment files
- auth or secret-like files
```

```
Do not create placeholder features, fake login pages, or speculative
abstractions.

EXECUTION PROCESS

1. Inspect the directory and tool versions.
2. Create a short internal implementation checklist.
3. Scaffold the smallest suitable project.
4. Implement all P0 features.
5. Generate the sample PDFs.
6. Run the tests, lint, and production build.
7. Fix all failures.
8. Start the app and perform a local smoke test.
9. Report exact commands, completed features, test results, and any
intentionally deferred P1 work.

Do not stop to ask product questions. Make reasonable implementation
decisions that maximize demo reliability. Do not claim completion until the
production build succeeds.
```

PROMPT_01_BOOTSTRAP.md

```
Work only in the current directory. Do not assume or create a parent path.
Preserve existing files.

Create the initial NodeWitness localhost project using the smallest
reliable Next.js, React, and TypeScript setup. Initialize Git locally if
needed, but do not create a remote or push.

Build an English-only enterprise dashboard shell with:

- NodeWitness branding
- tagline: “Build AI agents anyone can understand, review, and control.”
- workspace: “Aurora Family Office — Client Mandate Compliance”
- runtime mode badge
- Monitoring Active badge
- left source/builder panel
- center graph area
- right inspector area
- findings/actions area
- audit timeline area
- Reset Demo and Export Audit controls

Use native CSS or CSS modules. Create a polished dark enterprise visual
system using deep navy, cyan, emerald, amber, red-orange, and violet.
Include accessible focus states and status text.

Do not implement authentication, databases, external services, or cloud
deployment.
```

```
Create a clean domain-state structure for one demo workspace. Keep the
number of files and dependencies small.

Add an English README with install and startup instructions.

Run lint and the production build. Fix all failures before reporting
completion.
```

PROMPT_02_FIXTURES_AND_ENGINE.md

```
Continue the existing NodeWitness project in the current directory. Inspect
existing work and preserve it.

Implement the deterministic RIA compliance domain engine and bundled demo
data.

Create two synthetic English IPS PDFs:

IPS v1:
- Aurora Family Office
- direct cryptocurrency exposure maximum: 10%

IPS v2:
- Aurora Family Office
- total cryptocurrency exposure maximum: 5%
- includes direct holdings and look-through exposure through funds, ETFs,
and derivatives

Synthetic portfolio:
- direct crypto: 2.0%
- indirect/look-through crypto: 6.2%
- total crypto: 8.2%
- largest issuer: 18.7%
- issuer maximum: 20.0%
- liquid assets: 6.4%
- liquidity minimum: 5.0%

Store the PDFs under public/demo and ensure stable page numbers and
evidence excerpts.

Implement pure deterministic functions for:

- policy version diff
- compliance evaluation
- affected-node selection
- recommendation generation
- audit-event creation

Expected results:
```

```
- IPS v1 crypto conclusion: Pass
- IPS v2 crypto conclusion: Review Required
- IPS v2 must identify both the 10% to 5% threshold change and the scope
expansion
- only the crypto branch is affected by the IPS update
- concentration and liquidity branches remain unchanged

Generate these revised-run outputs:

1. Primary remediation case proposal
2. Compliance memo proposal
3. Adviser notification draft proposal
4. Look-through-data validation proposal
5. Near-limit issuer concentration attention item

Use one normalized schema that can later accept either fixture or Live
Codex data.

Add the smallest useful automated tests for:
- v1 decision
- v2 decision
- version diff
- affected-node selection

Use minimal dependencies. Run the tests, lint, and production build. Fix
all failures.
```

PROMPT_03_GRAPH_SEARCH_AND_TRACE.md

```
Continue the existing NodeWitness project. Inspect the current
implementation first.

Implement the complete interactive workflow visualization using
@xyflow/react.

Required nodes:

- IPS source
- portfolio source
- source versioning
- policy extraction
- version diff
- crypto rule
- exposure calculation
- compliance evaluation
- parallel attention checks
- recommendations
- human approval gate
- remediation case
- compliance memo
```

```
- adviser notification

Required behavior:

- pan
- zoom
- fit-to-view
- node selection
- visible status styling
- animated affected edges
- expandable node inspector
- clear selected-node state
- clear Human Modified state
- clear Review Required state

Implement global keyword search across:
- labels
- descriptions
- IDs
- rule text
- inputs
- outputs
- evidence excerpts

Searching “crypto” must visibly highlight all matching nodes. Clicking a
result must center and open the node.

Implement separate node-level search inside the inspector. It must
highlight matches within inputs, outputs, evidence, and metadata.

The inspector must show:
- business summary
- status
- inputs
- outputs
- evidence with source version, page, and quote
- concise decision basis
- technical metadata
- run history
- correction field

Evidence links must open the bundled PDF at the relevant page using the
browser’s native PDF viewer. Do not build a custom PDF renderer.

Do not expose hidden chain-of-thought. Label the section “Decision Basis.”

Keep the UI polished and presentation-ready. Run lint and the production
build and fix failures.
```

PROMPT_04_MONITORING_AND_RERUN.md

```
Continue the current NodeWitness project and preserve existing behavior.

Implement the end-to-end source update and incremental re-evaluation story.

Initial state:
- IPS v1 is active
- crypto conclusion is Pass
- monitoring is visibly active

When the presenter uploads or triggers IPS v2:

1. Emit an internal source.version.changed event.
2. Add the new source version without overwriting v1.
3. Display the old and new rule side by side.
4. Highlight:
   - 10% to 5%
   - direct-only to total look-through exposure
5. Show exact source page and evidence excerpt.
6. Calculate affected nodes.
7. Mark only those nodes Queued and then Running.
8. Animate only affected edges.
9. Preserve unaffected node run IDs and statuses.
10. Produce Review Required for the crypto rule.
11. Generate the remediation case and parallel recommendations.
12. Record every step in the audit timeline.

Use a deterministic state machine and short, reliable presentation timings.
The update must complete within about five seconds in Demo Mode.

Provide:
- source version history
- last monitoring check timestamp
- change detected indicator
- re-run progress
- Reset Demo

Reset Demo must return the entire application to the original IPS v1
passing state.

Do not add external connectors or infrastructure. Run tests, lint, and the
production build.
```

PROMPT_05_HUMAN_REVIEW_AND_ACTIONS.md

```
Continue the current NodeWitness project.

Implement the human governance and safe local execution layer.

Every recommendation must independently support:
```

```
- Approve
- Reject
- Edit
- View Evidence

Do not add batch approval.

The primary recommendation must be the remediation case. Show other
recommendations in parallel.

Editing requirements:
- preserve the original text
- store the edited text
- set decision status to Edited
- create an audit event
- allow the edited item to be approved later

Key decision nodes must include “Suggest a correction.”

Submitting a correction must:
- preserve the original node output
- record the human instruction
- mark the node Human Modified
- identify downstream nodes
- visually re-run only downstream nodes
- create a revised output
- record the complete event sequence

Approved actions may only perform these local operations:
1. Create a visible local remediation case.
2. Generate a downloadable Markdown compliance memo.
3. Generate an adviser notification draft.

Rejected or pending actions must not execute.

The remediation case should include:
- case ID
- title
- severity
- status
- client
- source versions
- violated rule
- observed exposure
- threshold
- evidence
- approved description
- owner placeholder
- due-date suggestion
- complete audit references

Implement audit JSON export and compliance memo download.

Persist useful demo state in localStorage if it can be done simply and
```

```
safely. Reset Demo must clear that state.

Run tests, lint, and production build. Fix failures.
```

PROMPT_06_CODEX_MCP_RUNTIME.md

```
Continue the current NodeWitness project. Do not disrupt the deterministic
Demo Mode.

Implement a minimal optional Live Codex runtime using the local Codex MCP
server.

Requirements:

- Use @modelcontextprotocol/sdk.
- Start or connect to `codex mcp-server` over stdio from server-only
Node.js code.
- Use the documented `codex` tool.
- Use `codex-reply` only if continuation is actually needed.
- Do not use the OpenAI API.
- Do not require OPENAI_API_KEY.
- Do not read, expose, copy, or commit ~/.codex/auth.json.
- Assume the user authenticates separately through the Codex CLI.
- Keep the integration localhost-only.
- Use read-only sandboxing and approval policy never.
- Never allow PDF content to cause shell or filesystem actions.
- Request strict JSON matching the existing normalized fixture schema.
- Validate and normalize all responses.
- Set a bounded timeout.
- Reuse one MCP process where safely practical.
- Clean up the process when the application terminates.
- Add a lightweight health/preflight endpoint.

Runtime behavior:

- If preflight succeeds, display Live Codex.
- If preflight fails, display Demo Mode.
- If a live request fails, times out, or returns invalid data:
  - preserve current UI state
  - switch to deterministic fixtures
  - display a concise English notification
  - keep the demo moving
- The runtime mode badge must always be accurate.
- Add a manual mode selector or retry control if it remains simple.

Live Mode should support at least:
1. Converting the natural-language workflow request into the existing graph
schema.
2. Producing a structured compliance assessment from the synthetic rule and
portfolio facts.
```

```
3. Revising a node after a human correction.

Do not expose hidden reasoning. Request:
- conclusion
- concise decision basis
- evidence references
- recommendations
- structured metadata

Do not introduce the OpenAI Agents SDK, a separate backend, a provider
abstraction framework, or an API-key flow.

Update the README with:
- Codex CLI prerequisite
- login/preflight instructions
- Live Mode behavior
- Demo Mode fallback
- troubleshooting
- security warning for the Codex auth file

Verify both conditions:
1. The complete demo works when Codex is available.
2. The complete demo works when the codex command is unavailable.

Run tests, lint, and the production build. Fix all failures.
```

PROMPT_07_DEMO_HARDENING.md

```
Act as the final senior engineer, product designer, and hackathon demo
reviewer for the current NodeWitness repository.

Do not add new product scope.

First inspect the full application and run:
- tests
- lint
- production build

Fix all failures and obvious runtime errors.

Then test the complete presentation path:

1. Open the RIA workspace.
2. Generate the workflow from the prefilled natural-language request.
3. Load IPS v1.
4. Confirm Pass.
5. Search globally for “crypto.”
6. Open a node.
7. Search for “5%” or “look-through” inside the node when applicable.
8. Open the PDF evidence page.
```

```
9. Trigger IPS v2.
10. Confirm the diff.
11. Confirm only affected nodes re-run.
12. Confirm Review Required.
13. Edit one recommendation.
14. Approve the remediation case.
15. Reject another recommendation.
16. Confirm only approved actions execute.
17. Generate the memo and notification draft.
18. Inspect the audit timeline.
19. Export audit JSON.
20. Reset the demo.
21. Repeat in Demo Mode with Codex unavailable.

Polish only what materially improves the presentation:

- loading states
- empty states
- failure states
- layout overflow
- node readability
- inspector readability
- search highlighting
- status labels
- update animations
- button feedback
- mode badges
- evidence navigation
- Reset Demo reliability

Check laptop viewport sizes, especially 1440x900 and 1280x800.

Ensure every user-facing string is English.

Remove:
- dead code
- unused dependencies
- placeholder pages
- speculative abstractions
- fake authentication
- unused configuration

Security review:
- no secrets
- no API keys
- no auth files
- no real customer data
- localhost binding
- uploaded content treated as untrusted
- no model-triggered shell execution

Update the README with the final exact setup command, startup command, demo
script, fallback procedure, and known limitations.
```

```
Do not report completion until the production build succeeds and the
deterministic full demo path is verified.

Finish with a concise report containing:
- files/features completed
- commands executed
- test/build results
- exact demo startup steps
- Live Mode preflight steps
- emergency Demo Mode procedure
- intentionally deferred roadmap items
```

Presenter emergency procedure

Before the live pitch:

1. Start the application.
2. Reset the demo.
3. Confirm IPS v1 shows `Pass`.
4. Test global search for `crypto`.
5. Trigger IPS v2 once.
6. Verify `Review Required`.
7. Reset again.
8. Check the runtime badge.
9. If Live Codex is slow or unstable, switch to `Demo Mode` before presenting.
10. Keep the app open on the initial workspace screen.

Demo Mode is not a compromised version of the presentation. It is the deterministic execution path that guarantees the complete product story. Live Codex should be treated as an enhancement, not a single point of failure.